Introduction to the Theory of Computation

Lecture 12. Universal Turing Machine

Neo (Yuan) Li

Oct 2026

Fudan University

yuan_li@fudan.edu.cn

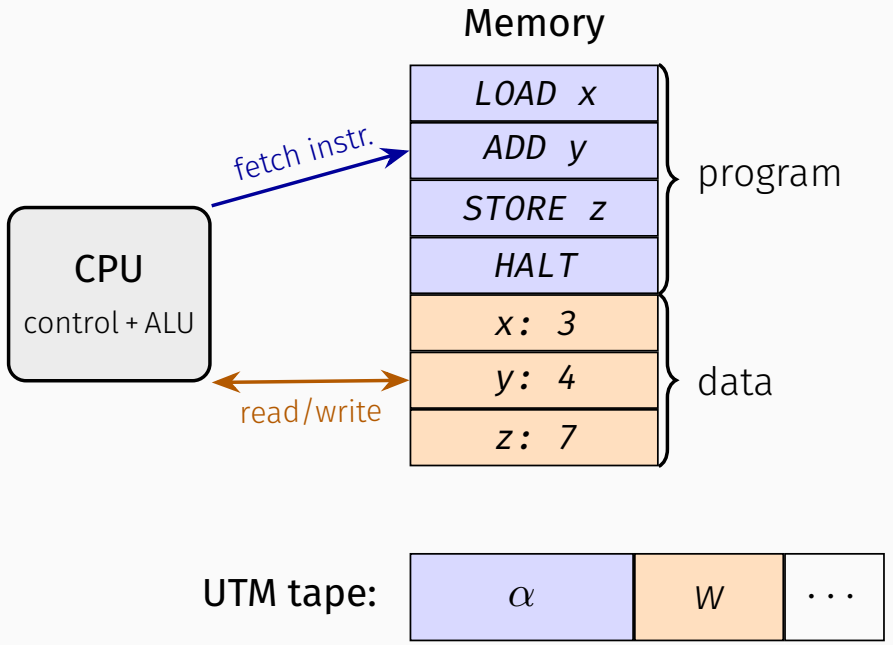
Why a Universal Turing Machine?

1. **One machine** runs every program — no special hardware per algorithm.
2. Encoding machines as strings lets us **prove computability results**.
3. Origin of the **stored-program computer** (von Neumann architecture, 1945).

Stored-Program Computers

Stored-program computer: instructions and data share *one* memory.

- A program is just *data* interpreted as instructions.
- Basis of the **von Neumann architecture** (1945).
- UTM = *model*; computer = *realization*.



Which Devices Are (Not) Universal?

Not universal	Universal
Calculators	Laptops / desktops
Traffic light controllers	Smartphones
Elevator controllers	

Left: fixed logic. **Right:** runs **any** program you load.

Encoding a Turing Machine as a String

Assume our encoding scheme satisfies:

- Every string $\alpha \in \{0,1\}^*$ represents **some** TM, denoted M_α. (If α is invalid, M_α always rejects.)
- Every TM is represented by **infinitely many** strings.

Example: the source code of a TM at *https://turingmachine.io/* or *https://turingmachinesimulator.com/*.

The Universal Turing Machine Theorem

Theorem 1
There is a 2-tape TM U such that for all $x, \alpha \in \{0,1\}^*$,

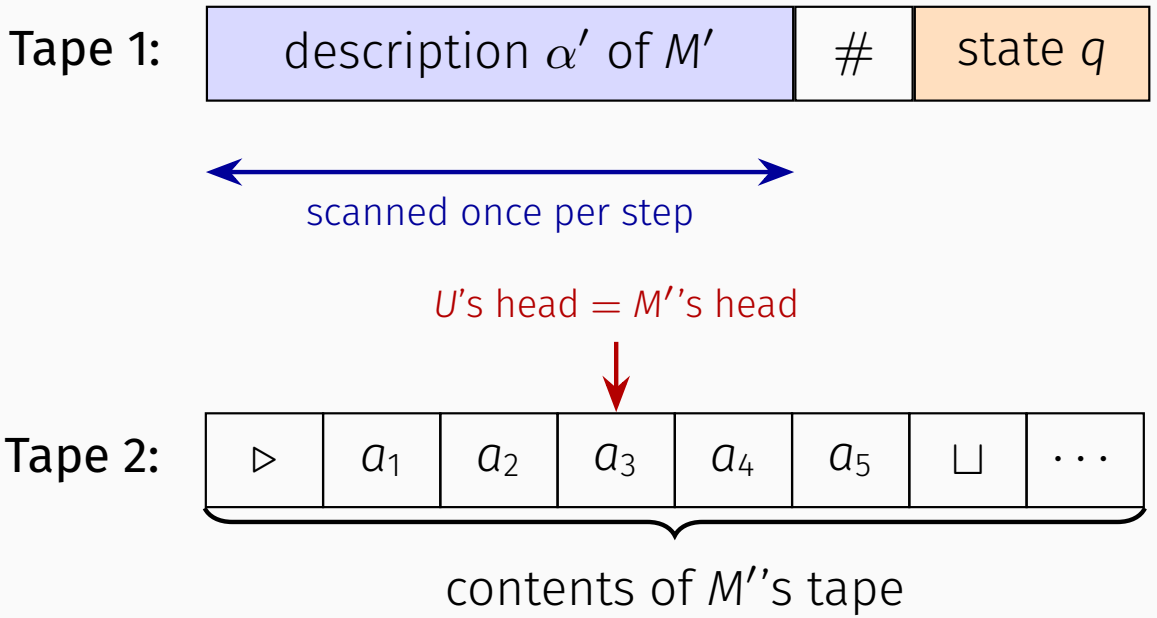
$$U(x, \alpha) = M_\alpha(x)$$

(same outcome: accept, reject, or run forever). Moreover, if M_α halts on x within T steps, then U halts within $O_\alpha(T \log T)$ steps.

Hennie & Stearns, "Two-tape simulation of multi-tape Turing machines," *JACM* 13(4), 1966.

Warm-Up: A Simpler UTM in $O_\alpha(T^2)$

Idea: convert M_α into a **single-tape** TM M' (earlier lemma), then simulate M' using U's two tapes.



Initially tape 1 holds $\langle x, \alpha \rangle$; U copies x to tape 2 and replaces α by α'.
No output tape: U just accepts or rejects.

Warm-Up: Proof

Setup (cost depends only on α, plus $O(n)$): copy x to tape 2; on tape 1, replace α by α' and the start state of M'.

Each step of M':

1. Read the symbol s under the tape-2 head.
2. Scan α' on tape 1 for the entry (q, s).
3. Write the new symbol, move the tape-2 head, update q.

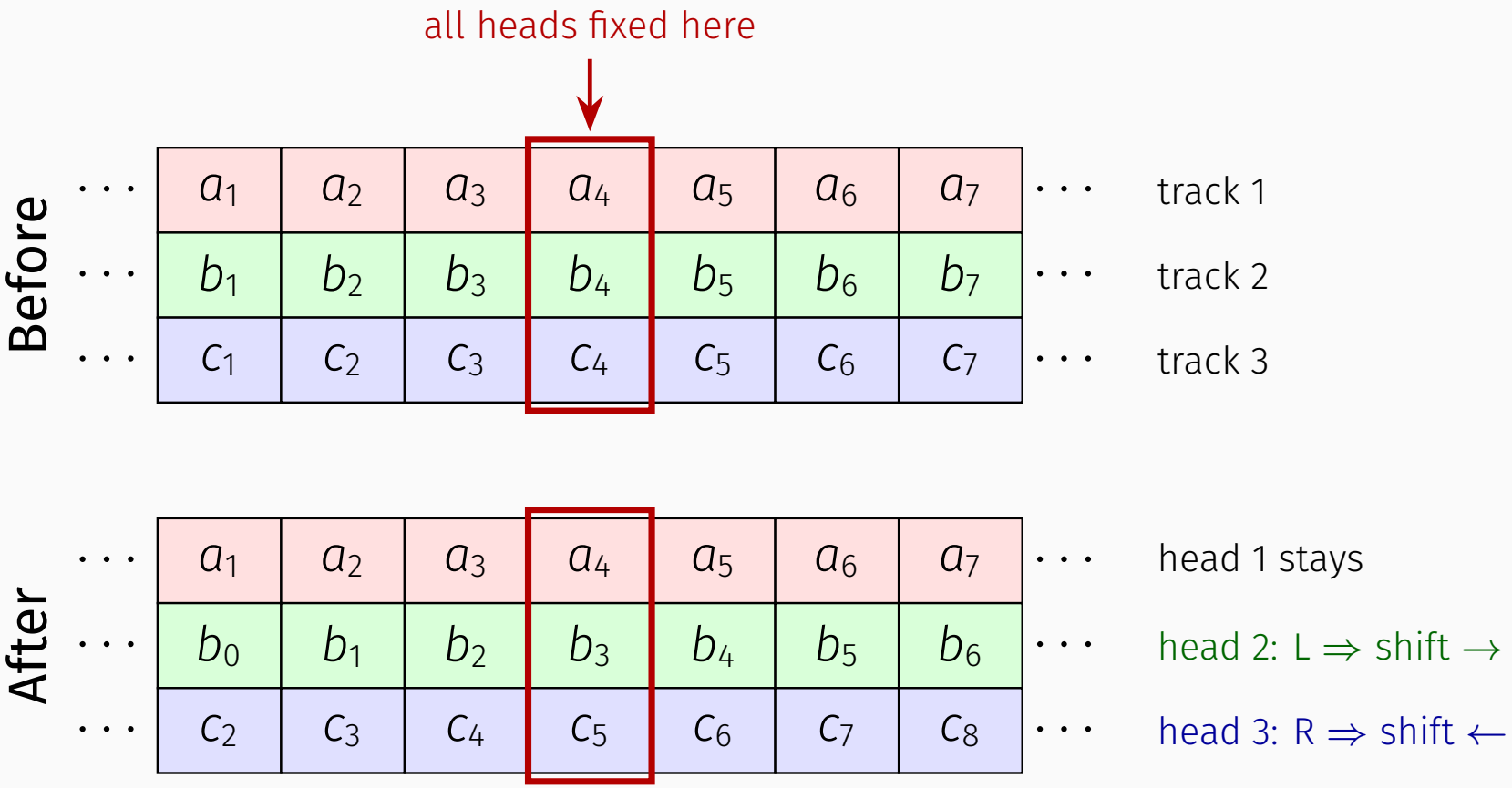
Cost: $O_\alpha(1)$

Total: M' runs $O(T^2)$ steps (lemma), so U runs $O_\alpha(T^2)$ steps (assuming $T \geq n$).

Towards $T \log T$: Tracks Instead of Tapes

- Tape 1: α and current state, as before.
- Tape 2: all k tapes of M as k **tracks** (alphabet Γ^k).
- All k simulated heads sit at one **fixed** cell.
- Key trick: to move a head, **shift its track** instead.

Compressing k Tapes into One



One tape over Γ^k; one head move = shift one track by one cell.

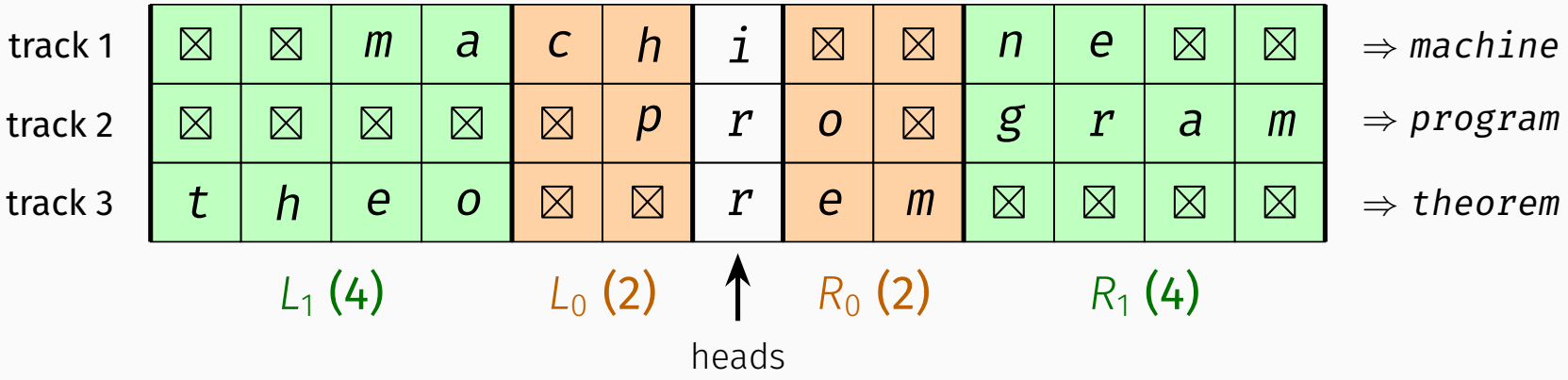
Dividing the Tape into Zones

Around the fixed head cell (the **origin**), tape 2 is split into zones

$$\cdots\ L_1\, L_0\ \ \text{origin}\ \ R_0\, R_1\ \cdots, \qquad |L_i| = |R_i| = 2^{i+1}.$$

- Zones up to index $\lceil \log T \rceil$ suffice.
- Filler symbol $\boxtimes$ = "empty slot" (not M's blank $\sqcup$).

Zone Layout: An Example



Drop ⊠ and read each track left to right $\Rightarrow$ M's tape. Heads read $\mathtt{i}$, $\mathtt{r}$, $\mathtt{r}$.

Invariants

For each track, at all times:

- Each zone is **empty**, **half-full**, or **full**: 0, 2^i, or 2^{i+1} non-$\boxtimes$ cells.
- $L_i \cup R_i$ has exactly 2^{i+1} non-$\boxtimes$ cells: (full, empty), (empty, full), or (half, half).
- The origin always holds the symbol under M's head.

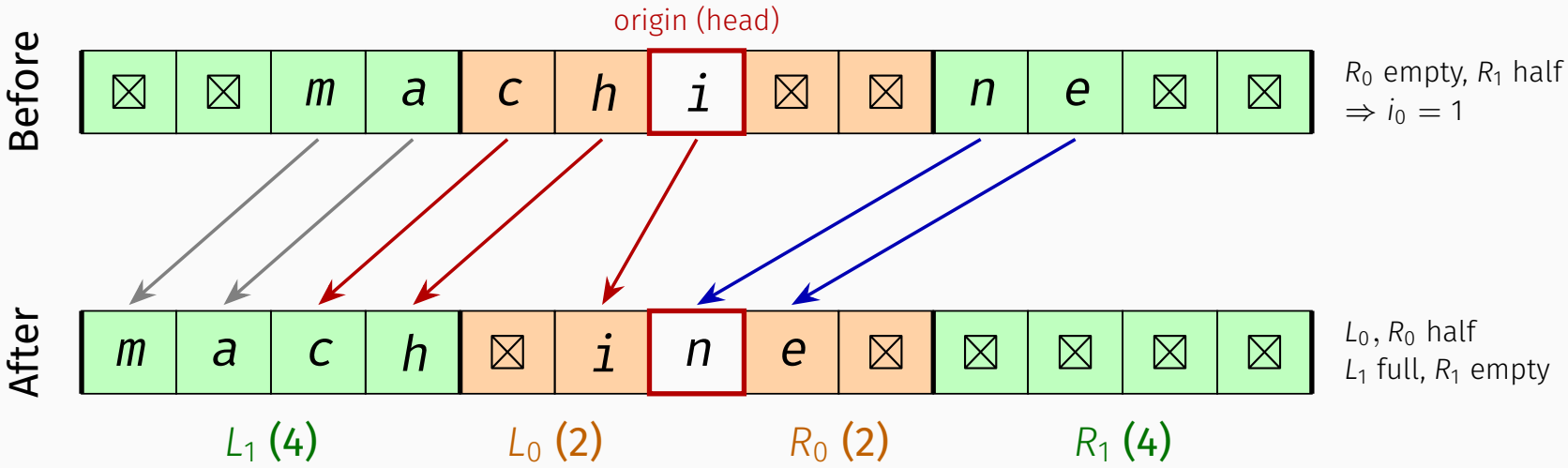
Performing a Left Shift

Left shift: a track's contents move one cell left.

1. Find the smallest i_0 with R_{i_0} non-empty. (So $R_0, \ldots, R_{i_0-1}$ are empty, $L_0, \ldots, L_{i_0-1}$ full.)
2. **Right:** leftmost symbol of $R_{i_0} \to$ origin; the next $2^{i_0} - 1$ fill $R_0, \ldots, R_{i_0-1}$ halfway.
3. **Left:** of the $2^{i_0+1} - 1$ symbols in $L_{i_0-1}, \ldots, L_0$ and the origin, the leftmost 2^{i_0} go into L_{i_0}; the rest fill $L_{i_0-1}, \ldots, L_0$ halfway.
4. Zones $0, \ldots, i_0 - 1$ are now half-full; R_{i_0} loses and L_{i_0} gains 2^{i_0} symbols. Invariants hold.

i_0 is the **index** of the shift; it costs $O(2^{i_0})$ steps.

Left Shift: An Example ($i_0 = 1$)



Right: $n \to$ origin, $e \to R_0$. Left: $c, h \to L_1$, $i \to L_0$.

Still reads *machine*; the head now sees n.

Amortized Cost: Key Lemma

Convention: initially every zone is half-full (pad with $\sqcup$).

Lemma 2
On each track, before the first shift of index $\geq i$, and between any two such shifts, there are at least $2^i - 1$ shifts of index $< i$.

Proof.
Let S = number of non-$\boxtimes$ cells in $R_0 \cup \cdots \cup R_{i-1}$ (capacity $2^{i+1} - 2$).

- Initially, or right after a shift of index $\geq i$: zones $0, \ldots, i-1$ are half-full, so $S = 2^i - 1$.
- A shift of index $\geq i$ needs $R_0, \ldots, R_{i-1}$ all empty (left shift: $S = 0$) or $L_0, \ldots, L_{i-1}$ all empty, i.e. R's all full (right shift: $S = 2^{i+1} - 2$).
- A shift of index $j < i$ changes S by exactly ± 1: it only rearranges zones $0, \ldots, j$, and exactly one symbol crosses the origin.

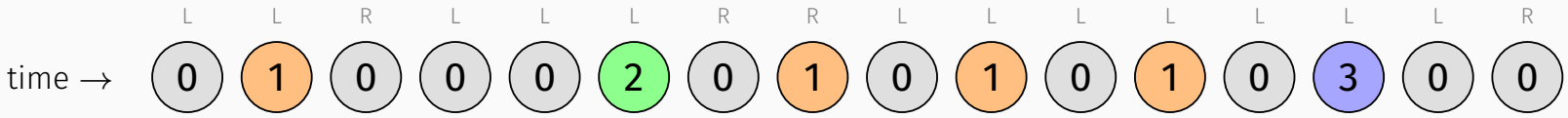
Moving S from $2^i - 1$ to 0 or to $2^{i+1} - 2$ takes at least $2^i - 1$ such shifts. $\square$

Amortized Cost: Key Lemma

Corollary 3

In T steps, each track has at most $T/2^i$ shifts of index i. In particular, every index is at most $\log(T+1)$.

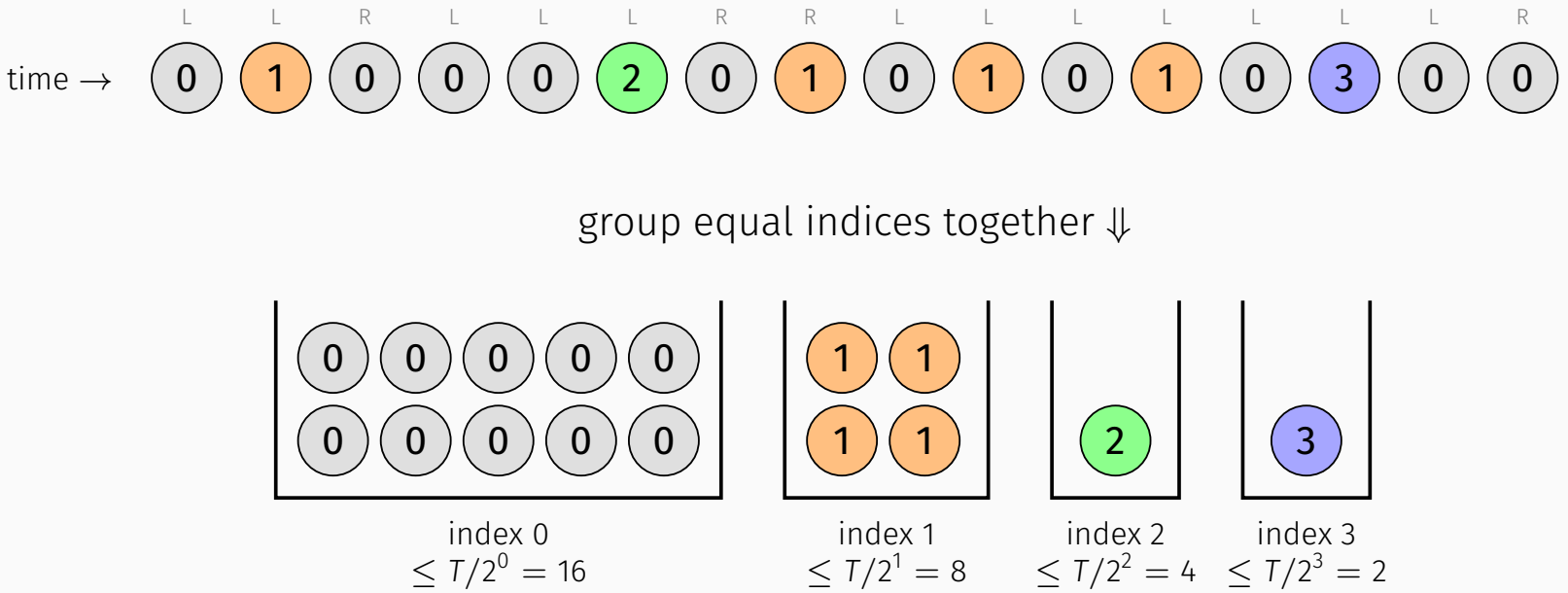
Double Counting: Balls and Bins



Each shift is a **ball** labeled with its index j; it costs 2^j.

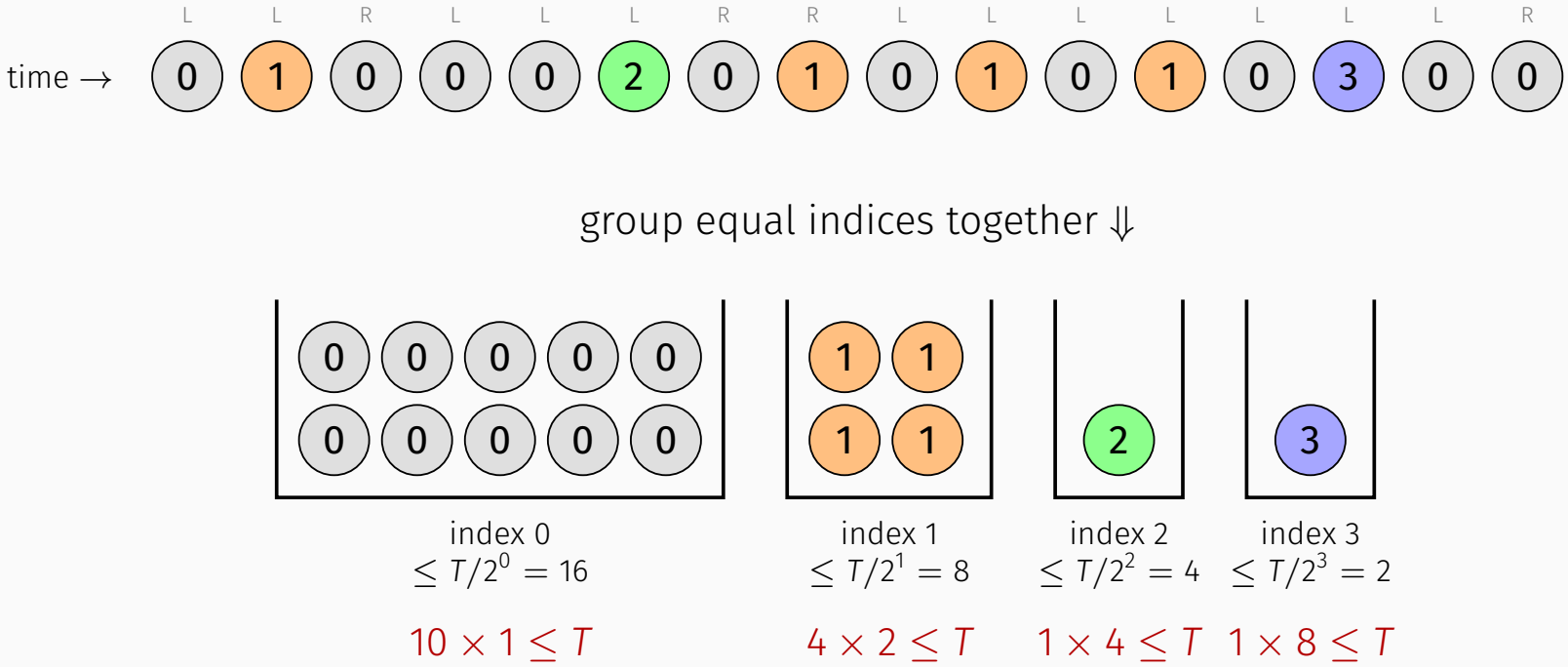
Here $T = 16$.

Double Counting: Balls and Bins



Same balls, sorted into **bins** by index. Lemma: bin j holds at most $T/2^j$ balls.

Double Counting: Balls and Bins



Bin j costs $\leq \frac{T}{2^j} \cdot 2^j = T$. $O(\log T)$ bins $\Rightarrow$ total $O(T \log T)$.

Amortized Cost: $O_\alpha(T \log T)$ Total

Double counting: add up the cost of all T shifts in two orders — by time, or grouped by index.

$$\sum_{t=1}^{T} O\big(2^{\mathrm{index}(t)}\big) = \sum_{j=0}^{\log(T+1)} O(2^j) \cdot \#\{t : \mathrm{index}(t) = j\}$$

$$\leq \sum_{j=0}^{\log(T+1)} O(2^j) \cdot \frac{T}{2^j} = \sum_{j=0}^{\log(T+1)} O(T) = O(T \log T).$$

That is per track; with k tracks, $k \cdot O(T \log T) = O_\alpha(T \log T)$.